Practical 1(A) — TCP Prime Number

1. `tcpClientPrime.java`

```
import java.net.*;
import java.io.*;

class tcpClientPrime {
   public static void main(String args[]) {
      try {
         Socket s = new Socket("localhost", 8000);
         BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

         System.out.print("Enter a number: ");
         int n = Integer.parseInt(br.readLine());

         DataOutputStream out =
            new DataOutputStream(s.getOutputStream());
         DataInputStream in =
            new DataInputStream(s.getInputStream());

         out.writeInt(n);
         System.out.println(in.readUTF());
         s.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

2. `tcpServerPrime.java`

```
import java.net.*;
import java.io.*;

class tcpServerPrime {
    public static void main(String args[]) {
        try {
            ServerSocket ss = new ServerSocket(8000);
            System.out.println("Server Started");

            Socket s = ss.accept();
            DataInputStream in =
                new DataInputStream(s.getInputStream());
            DataOutputStream out =
                new DataOutputStream(s.getOutputStream());
```

```
            int n = in.readInt(), i;
            for (i = 2; i <= n / 2; i++)
                if (n % i == 0) break;

            out.writeUTF(n > 1 && i > n / 2
                ? n + " is Prime" : n + " is not Prime");

            s.close();
            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Practical 1(B) – TCP Chat Application

3. ChatClient.java

```
import java.net.*;
import java.io.*;

class ChatClient {
    public static void main(String args[]) {
        try {
            Socket s = new Socket("localhost", 8000);
            BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in));
            BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(
                s.getOutputStream(), true);

            String msg;
            System.out.println("Type STOP to stop chatting");

            while (true) {
                System.out.print("Client Says: ");
                msg = br.readLine();
                out.println(msg);
                if (msg.equals("STOP")) break;
                System.out.println("Server Says: " +
in.readLine());
            }
            s.close();
```

```
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

4. ChatServer.java

```
import java.net.*;
import java.io.*;

class ChatServer {
    public static void main(String args[]) {
        try {
            ServerSocket ss = new ServerSocket(8000);
            System.out.println("Waiting for client...");
            Socket s = ss.accept();

            BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
            BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in));
            PrintWriter out = new PrintWriter(
                s.getOutputStream(), true);

            String msg;
            while ((msg = in.readLine()) != null) {
                if (msg.equals("STOP")) break;
                System.out.println("Client Says: " + msg);
                System.out.print("Server Says: ");
                out.println(br.readLine());
            }
            s.close();
            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Practical 1(A) — Prime Number

Compile (Terminal 1):

```
javac tcpClientPrime.java tcpServerPrime.java
```

Terminal 1 — Start server:

```
java tcpServerPrime
```

Terminal 2 — Start client:

```
java tcpClientPrime
```

Enter a number, e.g. 7.

Practical 1(B) — Chat Application

Compile (Terminal 1):

```
javac ChatClient.java ChatServer.java
```

Terminal 1 — Start server:

```
java ChatServer
```

Terminal 2 — Start client:

```
java ChatClient
```

Type a message in the client terminal. Reply from the server terminal. Type `STOP` in the client to end the chat.

Important: Stop the previous server with `Ctrl + C` before starting the next one.

```
Yes. Practical 2 has two parts: UDP Even/Odd and RPC Calculator.
I'll shorten both while keeping the same functionality and show
sample outputs.
```

The main trick is to remove unnecessary variables and comments, not the networking code your teacher expects to see.

Practical 2(A) — UDP Even or Odd

1. `udpClientEO.java`

```
import java.net.*;
import java.io.*;
```

```
class udpClientEO {
    public static void main(String args[]) throws Exception {
        DatagramSocket ds = new DatagramSocket(1000);
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter a number: ");
        byte b[] = br.readLine().getBytes();

        ds.send(new DatagramPacket(b, b.length,
            InetAddress.getLocalHost(), 2000));

        b = new byte[1024];
        DatagramPacket dp = new DatagramPacket(b, b.length);
        ds.receive(dp);

        System.out.println(new String(dp.getData(), 0, dp.getLength()));
        ds.close();
    }
}
```

2. `udpServerEO.java`

```
import java.net.*;

class udpServerEO {
    public static void main(String args[]) throws Exception {
        DatagramSocket ds = new DatagramSocket(2000);
        byte b[] = new byte[1024];

        DatagramPacket dp = new DatagramPacket(b, b.length);
        ds.receive(dp);

        int n = Integer.parseInt(
            new String(dp.getData(), 0, dp.getLength()));

        b = (n % 2 == 0 ? "Number is even" : "Number is odd")
            .getBytes();

        ds.send(new DatagramPacket(b, b.length,
            dp.getAddress(), dp.getPort()));
        ds.close();
    }
}
```

Practical 2(B) — Calculator Using Client-Server RPC

3. RPCClient.java

```
import java.net.*;
import java.io.*;

class RPCClient {
    public static void main(String args[]) throws Exception {
        DatagramSocket ds = new DatagramSocket();
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

        while (true) {
            System.out.print("Enter operation (add 3 4): ");
            byte b[] = br.readLine().getBytes();

            ds.send(new DatagramPacket(b, b.length,
                InetAddress.getLocalHost(), 12000));

            b = new byte[1024];
            DatagramPacket dp = new DatagramPacket(b, b.length);
            ds.receive(dp);

            System.out.println("Result = " +
                new String(dp.getData(), 0, dp.getLength()));
        }
    }
}
```

4. RPCServer.java

```
import java.net.*;

class RPCServer {
    public static void main(String args[]) throws Exception {
        DatagramSocket ds = new DatagramSocket(12000);
        byte b[] = new byte[1024];

        while (true) {
            DatagramPacket dp = new DatagramPacket(b, b.length);
            ds.receive(dp);

            String a[] = new String(dp.getData(), 0,
                dp.getLength()).split(" ");

            int x = Integer.parseInt(a[1]);
            int y = Integer.parseInt(a[2]);
            int r = 0;

            switch (a[0]) {
                case "add": r = x + y; break;
                case "sub": r = x - y; break;
                case "mul": r = x * y; break;
                case "div": r = x / y; break;
            }

            b = ("" + r).getBytes();
            ds.send(new DatagramPacket(b, b.length,
                dp.getAddress(), dp.getPort()));
        }
    }
}
```

1. Create the files

Practical2

`udpClientEO.java`

`udpServerEO.java`

`RPCClient.java`

`RPCServer.java`

You can keep all four files in the same folder.

2. Run the UDP Even/Odd program

Terminal 1 — compile both files:

```
javac udpClientEO.java udpServerEO.java
```

Terminal 1 — start the server:

```
java udpServerEO
```

Leave it running.

Terminal 2 — start the client:

```
java udpClientEO
```

3. Run the RPC calculator

Stop the UDP server with `Ctrl + C` when finished.

Terminal 1:

```
javac RPCClient.java RPCServer.java
java RPCServer
```

Terminal 2:

```
java RPCClient
```

Practical 3 — Multicast Socket Example

Aim: Multicast Socket Example

1. `BroadcastClient.java` — Receiver

```
import java.net.*;

public class BroadcastClient {
    public static final int PORT = 1234;

    public static void main(String args[]) throws Exception {
        MulticastSocket socket;
        DatagramPacket packet;
        InetAddress address;

        address = InetAddress.getByName("239.1.2.3");
        socket = new MulticastSocket(PORT);
        socket.joinGroup(address);

        byte[] data = new byte[100];
        packet = new DatagramPacket(data, data.length);

        for (;;) {
            socket.receive(packet);

            String str = new String(packet.getData());

            System.out.println("Message received from " +
                packet.getAddress() + "\nMessage is: " + str);
        }
    }
}
```

2. `BroadcastServer.java` — Sender

```
import java.net.*;

public class BroadcastServer {
    public static final int PORT = 1234;

    public static void main(String args[]) throws Exception {
        MulticastSocket socket;
        DatagramPacket packet;
        InetAddress address;

        address = InetAddress.getByName("239.1.2.3");
        socket = new MulticastSocket();
        socket.joinGroup(address);

        byte[] data = null;

        for (;;) {
            Thread.sleep(10000);
            System.out.println("Sending");

            String str = "This is TCS calling...";
            data = str.getBytes();

            packet = new DatagramPacket(
```

```
                data, str.length(), address, PORT);
            socket.send(packet);
        }
    }
}
```

Run in VS Code

Compile both files:

```
javac BroadcastClient.java BroadcastServer.java
```

Terminal 1 — Start receiver first:

```
java BroadcastClient
```

Terminal 2 — Start sender:

```
java BroadcastServer
```

Practical 4 — RMI-Based Application to Display Current Date and Time

Aim: To develop an RMI-based application program to display the current date and time.

1. `InterDate.java`

```
import java.rmi.*;

public interface InterDate extends Remote {
    public String display() throws Exception;
}
```

2. `ServerDate.java`

```
import java.rmi.*;
import java.rmi.server.*;
import java.util.*;

public class ServerDate extends UnicastRemoteObject
        implements InterDate {

    public ServerDate() throws Exception {
    }

    public String display() throws Exception {
        String str = "";
        Date d = new Date();
        str = d.toString();
        return str;
```

```
    }

    public static void main(String args[]) throws Exception {
        ServerDate s1 = new ServerDate();
        Naming.bind("DS", s1);
        System.out.println("Object registered");
    }
}
```

3. `ClientDate.java`

```
import java.rmi.*;
import java.io.*;

public class ClientDate {
    public static void main(String args[]) throws Exception {
        String s1;
        InterDate i1 = (InterDate) Naming.lookup("DS");
        s1 = i1.display();
        System.out.println(s1);
    }
}
```

How to run in VS Code

Save all three files in the same folder.

Terminal 1 — Compile

```
javac *.java
```

Terminal 2 — Start RMI registry

```
rmiregistry
```

Terminal 3 — Run server

```
java ServerDate
```

Terminal 4 — Run client

```
java ClientDate
```